विद्या ददाति विनयम्

Vidhyāsathi

An Honest, Grounded Study Companion
Project Report

Answer when grounded. Be honest when unsure. Escalate when it matters.

Programme	**B.Tech CSE (Data Science), Semester 5 Mini-Project**
Institution	**Chinmaya Vishwa Vidyapeeth (CVV), Ernakulam**
Team	**Ravishankar Varier R · Krishna N U**
Guide	**Dr. Soumya Haridas**
Duration	**June – October 2026**
Repository	**github.com/raaaviiiii/vidhyasathi**

Abstract

General-purpose AI chatbots answer from the whole internet, hallucinate confidently, drift from the syllabus and never admit uncertainty — a poor fit for students revising from their own course material. **Vidhyāsathi** ("knowledge companion") is a local, honesty-first study assistant built on Retrieval-Augmented Generation. It ingests a student's own notes in any format (PDF, slides, documents, spreadsheets and scanned or handwritten pages through OCR), answers **only** from that material with a citation to the exact slide or page, and uses a CRAG-inspired three-band confidence gate to decide whether to answer, answer with a warning, or refuse honestly. Beyond question answering it offers an adaptive **Teach Me** tutor, **Exam & Practice** question generation with grounded feedback, and **Ask a Helper**, which routes an unresolved doubt to one matched teacher whose answer is fed back into the student's knowledge base. A conversation layer resolves follow-ups, reactions, pushback and typing slips so students can talk naturally. The system runs entirely on a laptop (Llama 3.1 8B via Ollama, MiniLM embeddings, a cross-encoder re-ranker and Qdrant), keeps each student's data private and isolated, and is verified by 456 backend tests, 186 interface tests, live end-to-end checks against the real model (18/18 passed, 100% honest refusals, 75% judged accuracy) and an evaluation harness benchmarked against a BM25 baseline.

Contents

1. 1. The Problem
2. 2. The Idea — Vidhyāsathi ("Knowledge Companion")
3. 3. Objectives
4. 4. Literature Base (Zeroth Review)
5. 5. Technology Stack
6. 6. System Architecture
7. 7. The Pipeline, Step by Step
8. 8. The Honesty Engine — Three-Band Confidence Gate
9. 9. Talking Like a Person — The Conversation Layer
10. 10. Study Modes
11. 11. Ask a Helper — Human in the Loop (Phase 2)
12. 12. Accounts, Privacy & Security
13. 13. The Interface
14. 14. Evaluation
15. 15. Development Journey
16. 16. Key Challenges & How We Solved Them
17. 17. Vision vs. Delivered
18. 18. Limitations
19. 19. Future Work
20. 20. Live Demo Script (≈ 5 minutes)
21. 21. Conclusion
22. 22. References
23. Appendix — Key Numbers at a Glance

1. The Problem

- Students study from **their own material**: lecture slides, class notes, syllabus PDFs and handwritten pages.
- General AI chatbots answer from the whole internet, not from *this* course. They often:
- **hallucinate**, giving confident answers that are wrong,
- **drift from the syllabus**, so answers don't match what the teacher taught,
- **never admit uncertainty**, so a student can't tell a good answer from a guess,
- **leave no path to a human** when the student is still stuck.
- Handwritten and scanned notes are invisible to most tools.
- Cloud tools also send private study material to third-party servers.

The gap: students need an assistant that knows *their* notes, cites where every answer came from, says "I don't know" honestly, and hands a doubt to a real teacher when it can't help.

---

2. The Idea — Vidhyāsathi ("Knowledge Companion")

A **local, honesty-first study assistant** built on Retrieval-Augmented Generation (RAG):

1. **Grounded:** it answers **only** from the student's own uploaded material, and every fact carries a citation such as *[2. NETWORK MODELS · slide 4]*.
2. **Honest:** a three-band **confidence gate** decides whether to answer, to answer with a warning, or to refuse.
3. **Teaches:** besides Quick Answers it has an adaptive **Teach Me** tutor, plus **Exam & Practice** for question generation and answer feedback.
4. **Escalates: Ask a Helper** sends a doubt to *one* matched teacher or senior. Their reply is added to the student's material, so the same question is answered from the notes next time.
5. **Private and local:** the language model, embeddings and database all run on the student's own machine.

---

3. Objectives

#	Objective	Status
1	Answer questions grounded in the student's own material, with citations	✅ Done
2	Refuse honestly when the material doesn't cover a question	✅ Done (100% honest refusals in live tests)
3	Ingest any format, including scanned and handwritten notes (OCR)	✅ Done (PDF, DOCX, PPTX, XLSX, CSV, TXT, MD, images)
4	Adaptive teaching (Teach Me) with conversation memory	✅ Done
5	Exam practice: generate questions and give feedback on answers	✅ Done
6	Escalate uncertain doubts to a human helper; feed answers back	✅ Done (Phase 2)
7	Multi-user web app with private, isolated material per student	✅ Done
8	Measurable quality: an evaluation harness against a baseline	✅ Done (Phase 2)
9	Natural, human-like conversation (follow-ups, typos, small talk)	✅ Done (final phase)
10	Fine-tuning the model on course data	⏳ Future work

4. Literature Base (Zeroth Review)

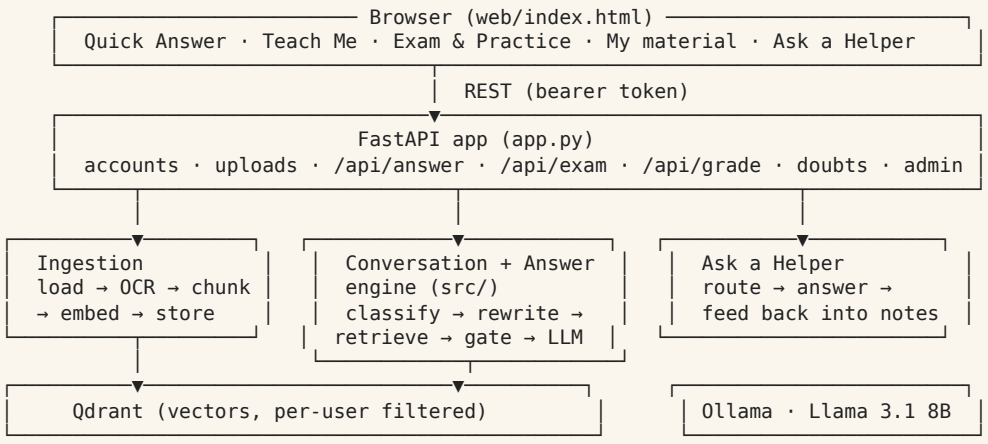
Eight 2024–2025 papers shaped the design:

Paper	Venue / ID	What we took from it
Agentic RAG Survey (Singh et al., 2025)	arXiv:2501.09136	Overall architecture: retrieval as a tool the assistant reasons over
CRAG: Corrective RAG (Yan et al., 2024)	arXiv:2401.15884	A *retrieval evaluator* checks whether retrieved passages actually answer the question → our **groundedness gate**
Adaptive-RAG (Jeong et al., NAACL 2024)	arXiv:2403.14403	Adapt the strategy to the question → **size-aware retrieval**, overview vs pointed questions, skipping checks on confident hits
Graph RAG (Edge et al., 2024)	arXiv:2404.16130	Whole-corpus "global" questions need a different path → our **overview mode** ("explain all concepts in module 3")
RAGChecker (Ru et al., NeurIPS 2024)	arXiv:2408.08067	Evaluate retrieval and generation separately → our **evaluation harness** (MRR, Recall, judged accuracy)
Hallucination Survey (Huang et al., ACM TOIS 2024)	arXiv:2311.05232	Why LLMs hallucinate; refusal and citation as mitigations → **honest refusal + mandatory citations**
LLMs for Education Survey (Wang et al., 2024)	arXiv:2403.18105	Tutoring patterns: one idea at a time, check understanding → **Teach Me** design
Short Answer Grading with RAG (Chu et al., 2025)	arXiv:2504.05276	Grade against reference material, not model opinion → **grounded feedback** in Exam & Practice

5. Technology Stack

Layer	Technology	Why
Language model	**Llama 3.1 8B** via **Ollama** (local)	Runs fully offline on a laptop; no data leaves the machine
Embeddings	**all-MiniLM-L6-v2** (sentence-transformers, 384-dim)	Fast, small and accurate for semantic search
Re-ranking	**ms-marco-MiniLM-L-6-v2** cross-encoder	Re-orders retrieved passages by true relevance
Vector database	**Qdrant** (Docker server)	Fast filtered vector search; multi-process safe
Backend	**Python, FastAPI, uvicorn**	Clean REST API, input validation with Pydantic
Auth & data	**SQLite**, PBKDF2-SHA256 (100k iterations) password hashing, bearer session tokens	Simple, secure, local
OCR	Vision-model OCR with caching	Reads scanned and handwritten pages once, then caches
Web fallback	DuckDuckGo search + trafilatura	Optional last resort, always labelled as web content
Frontend	Single-page HTML/CSS/JS, marked.js, KaTeX	Markdown and maths rendering, no build step
Design	**Figma**: warm-dark "manuscript" theme with Indian motifs (lotus mandala, Devanagari greeting)	A distinct, culturally rooted identity
Sharing	**Cloudflare Tunnel**	A public demo link without a server
Testing	pytest, JSDOM + node:test, live end-to-end checks	450+ backend tests, 180+ UI tests
Hardware	MacBook Pro M4 Pro (primary), Windows RTX 3050 Ti (GPU work)	

6. System Architecture



Project layout:

```
vidhyasathi/
├── app.py · start.sh · stop.sh · README.md
├── src/       engine: ingest, chunk, embed, store, retrieve, rerank, answer,
│              converse, exam, grade, doubts, eval, ocr, websearch
├── web/       index.html (the whole UI)
├── scripts/   vcheck autopilot, live smoke test, real-notes evaluation
├── tests/     backend + UI test suites
├── docs/      QA spec, test reports, this report
└── private/   🔒 accounts, uploads, knowledge base, logs, secrets (never committed, owner-only)
```

7. The Pipeline, Step by Step

```
Upload → Load (+OCR) → Chunk → Embed → Store ⟶ Understand → Retrieve → Re-rank → Gate → Answer → Cite
```

1. **Load:** a loader for each format (PDF pages, PPTX slides, DOCX headings, XLSX/CSV rows, images). Every piece keeps a **locator** (`slide 4`, `p12`) for citations.
2. **OCR fallback:** pages with too little text (under 20 characters) are treated as scanned and read by a vision model. A blur check routes difficult pages to a stronger model. Results are **cached** so a page is never paid for twice.
3. **Chunk:** 800-character chunks with a 150-character overlap.
4. **Embed:** MiniLM turns each chunk into a 384-dimensional vector.
5. **Store:** Qdrant stores the vectors with payload: user, document, citation label, locator and version. Re-uploads are **atomic**: the new version is written before the old one is retired.
6. **Understand the message (conversation layer):** classify, rewrite follow-ups, fix typos (see §9).
7. **Retrieve:** the top-15 candidates by cosine similarity, **filtered to this student only**.
8. **Re-rank:** the cross-encoder re-orders them (weight 0.7) and keeps the best 5. Small knowledge bases (30 chunks or fewer) skip this and use the whole document.
9. **Confidence gate:** decide whether to answer at all (see §8).
10. **Answer:** Llama 3.1 writes the reply using only the passages, citing each fact as `[source · locator]`.

---

8. The Honesty Engine — Three-Band Confidence Gate

Inspired by **CRAG**: check that retrieval is good *before* generating, so the model never invents an answer from weak passages.

Retrieval score (cosine)	What happens	Label shown
≥ 0.55 (strong)	Trust retrieval and skip the extra check, which **halves latency**	`grounded`
0.30 – 0.55 (middle)	An LLM **groundedness check** asks whether these passages actually cover the question, fully or partly	`grounded` or `tentative`
< 0.30 (weak)	Refuse honestly without calling the model	`not in material`
Answer score below 0.45	The answer is shown with a warning and an **Ask a Helper** button	`tentative`

Extra honesty rules: - **Mandatory citations:** every fact links back to a slide or page, shown as clickable chips. - **No invented reasons:** if the notes don't explain *why*, the model must say "*Your notes don't give the reason for that.*" - **Refusal with direction:** a refusal lists the closest topics the notes *do* cover, as one-click buttons. - **Web fallback (optional, off by default):** used only after a refusal and always tagged `[web · site]`. A scrubber strips any fake notes-style citation, so web content can never be passed off as the student's own notes. - **OCR caveat:** answers whose equations came from handwriting carry a "double-check this" note.

---

9. Talking Like a Person — The Conversation Layer

Real students don't type perfect, self-contained questions. They type "teach me it", "explain each concept in detail", "ok", "osimodel". The conversation layer (`src/converse.py`) works out what a message **means** before searching.

Student types	Before	Now
"teach me it" (after a question)	Searched literally, refused (score 0.16)	Rewritten as "Teach me guided transmission media" and taught
"explain each concept in detail"	Refused (score 0.41)	Reads the **whole named module** and walks through every concept
"hi", "thanks", "what can you do?"	"I don't have that in the material."	Friendly reply that names topics from the student's notes
"what files do I have?"	Refused	Lists the documents and their main topics
"ok / ahh yes it does" (in a lesson)	Repeated the same explanation	Moves on to the **next idea**
"no / I don't get it"	—	Re-explains the same idea more simply, with an example
"something different was taught in class"	Refused	Re-checks the notes, owns up to anything unsupported, suggests Ask a Helper
"explain the osimodel fully"	Refused (score 0.50)	Fixed to "osi model" against the notes' vocabulary, then answered
"what's on slide 4 of introduction?"	Vague search	Recalls exactly that slide
"give questions based on that" (Exam & Practice)	Refused	Practice set on the topic being discussed
"see both the materials, give an overview"	Refused (score 0.09)	Walks through every uploaded document

How it works: 1. **Classify** each message: small talk / understood / didn't understand / continue / pushback / follow-up / new question. 2. **Rewrite follow-ups** into a standalone question using the conversation, with guards: a rewrite that invents topics, or that answers instead of rewriting, is rejected. 3. **Fix typing slips** against the notes' own vocabulary: joined words ("datalink" → "data link"), swapped letters ("cabel" → "cable") and single typos ("satallite" → "satellite"). Real words, short words and word forms are never changed, so an off-topic question is never "corrected" into an on-topic one. 4. **Remember meaning:** each turn stores what it was understood as, so "my earlier question" resolves correctly many turns later. 5. **Anti-repetition:** the model sees its whole last reply, plus an end-of-message checklist (one idea at a time, never repeat, cite the notes). 6. **Transparency:** every follow-up shows a small line saying what it was understood as.

10. Study Modes

Quick Answer

- Direct, cited answers whose length matches the question.
- **Overview questions** ("summarise module 2", "explain all concepts") get a structured, cited tour of the whole document.

Teach Me

- A one-to-one tutor: **one idea at a time**, plain language, worked examples and everyday analogies.
- Ends every step with a question that checks understanding, and adapts to the reply (move on, slow down, or re-explain).
- Strict: a new topic that isn't in the notes gets an honest refusal, not a deflection.

Exam & Practice

- **Question generation:** practice questions on any topic in the notes, each with a cited model answer.
- **Grade my answer:** feedback under four headings (Correct · Missing · Incorrect · Suggestion), checked against the notes rather than the model's opinion. It is feedback, not a mark.

---

11. Ask a Helper — Human in the Loop (Phase 2)

Turns the thesis "*escalate when it matters*" into a feature.

1. When an answer is **tentative** or **refused**, an **Ask a Helper** button appears.
2. Before sending, the student sees **exactly who** it will go to: the best-matched helper (name and subjects), with the option to pick another. *Only that one person sees it.*
3. **Routing:** semantic similarity between the doubt and each helper's listed subjects, plus a keyword bonus, minus a small load penalty so work is spread out.
4. The helper answers from their **Helper inbox**.
5. **Feedback loop:** the answer is indexed into *that student's* knowledge base as "Helper answers · doubt N". Next time, Vidhyāsathi answers from it with a citation, so escalations fall over time.
6. **Owner tools:** the app owner makes accounts helpers, assigns subjects, and sees open and answered counts. With no helpers set up, doubts go to the owner.
7. **Privacy:** students never see helpers' emails, and helpers see only the student's first name.

---

12. Accounts, Privacy & Security

- **Per-user isolation:** every chunk is tagged with its owner. Retrieval, counts, exams, grading and deletes are all filtered by user, so one student can never see another's notes.
- **Authentication:** salted PBKDF2-SHA256 passwords and bearer session tokens tied to the account. Account IDs are never reused after a deletion.
- **Login protection:** lockout per email and client IP after repeated failures.
- **Safe uploads:** file names sanitised (no path traversal, hidden files or invisible characters), size limits, supported types only.
- **Prompt-injection guard:** a forged "system" message in the chat history is dropped before reaching the model.
- **Private folder:** accounts, uploads, the knowledge base, OCR text, logs and secrets live in `private/`. It is owner-only (chmod 700) and never committed to Git. Leftover files from the old layout migrate there automatically.
- **Local by design:** the LLM and embeddings run on the machine; nothing is sent to a cloud AI unless web fallback or OCR is used.

13. The Interface

- Designed in **Figma**, then built as a single page.
- **Warm-dark "manuscript" theme:** saffron and gold accents, a lotus mandala, *नमस्ते* greeting, and the motto *विद्या ददाति विनयम्* ("knowledge gives humility").
- Mode chips (Quick Answer · Teach Me · Exam & Practice) and a web-fallback toggle.
- Answers render **Markdown and maths** (KaTeX), with citation chips, a confidence badge, a score and a sources line.
- Sidebar: Current chat · Recent chats · My material · Practice sets · Trust & citations · My doubts · Helper inbox.
- Settings: default mode, web-fallback default, manage material, change password, delete account, and owner tools.
- Shared with anyone through the public tunnel link.

---

14. Evaluation

Evaluation harness (`src/eval.py`, inspired by RAGChecker)

- Automatically **drafts a test set** from the student's own notes, plus off-topic "negative" questions.
- **Retrieval metrics:** MRR@10 and Recall@5, compared against a **BM25 keyword baseline**.
- **Answer accuracy:** an LLM judge compares each answer with its source passage.
- **Honesty:** the refusal rate on negatives (should be 100%) and the false-refusal rate on answerable questions.
- **Threshold sweep:** recommends gate thresholds from real data.
- Runs on a **temporary copy** of the notes, so real data is never touched.

Live results (real Llama 3.1 8B on the MacBook Pro M4 Pro)

Metric	Result
Live end-to-end checks	**18 / 18 passed**
Honest refusal on off-material questions	**100%**
Judged answer accuracy	**75%** (20 questions)
Typical response time	**≈ 2 – 4 seconds**

Automated testing ("a team of testers")

- **3 structured QA rounds** against a written spec (`docs/qa/SPEC.md`): engine, ingestion and storage, API security, frontend, regression, Phase 2.
- **453 backend tests** (pytest, against a real Qdrant server, with a scripted stand-in model).
- **186 UI tests** (JSDOM): rendering, XSS safety, chats, modes, doubts.
- **Live smoke test** with ~30 real-model checks, including the conversation cases: follow-ups, "yes it does" not repeating, pushback, typos.
- `vcheck` **autopilot:** one command starts everything, runs every suite plus the live check and the evaluation, and writes a summary.

---

15. Development Journey

Phase	When	What was built
0. Research & design	Jun 2026	Problem framing, 8-paper literature review, name (*Vidhyāsathi*), thesis
1. Core pipeline (CLI)	Jun – Jul	Multi-format ingestion, OCR with caching and cost log, chunk/embed/store, retrieve + re-rank, size-aware context
2. Honesty & modes	Jul	Three-band confidence gate with CRAG check, Teach Me with memory, web fallback with anti-laundering scrub, exam generation, grading
3. Web application	Jul	Figma design, FastAPI backend, single-page UI, accounts, uploads, modals, Markdown and KaTeX
Zeroth review	5 Aug	16-slide deck, sources deck and PDF
4. Multi-user & speed	Aug – Sep	Per-user isolation, settings, overview questions, skipping the check on confident hits (half the model calls)
5. Deployment	Early Oct	Cloudflare public link; fixed uploads that never indexed; moved Qdrant to a Docker server
6. QA rounds	Oct	3 test rounds; every bug found was fixed (session auth, atomic re-index, safe uploads, isolation, honest gating)
7. Phase 2	Oct	Ask a Helper with the feedback loop; evaluation harness
8. Production polish	15 Oct	One-command start (`vstart` / `vstop` / `vcheck`), private-data reorganisation, conversation layer, typo handling, helper routing transparency

16. Key Challenges & How We Solved Them

Challenge	Root cause	Solution
Uploaded files never got indexed	Ingestion ran in a subprocess that only printed a report, and embedded Qdrant held a file lock	Moved ingestion into the server process (`ingest_file()`)
"Not indexed" and score 0.00 after deployment	**Embedded Qdrant is single-process**: writes from upload requests were invisible to query requests	Ran **Qdrant as a Docker server**; the app connects by URL
Maths caveat firing on every answer	The regex matched any single `$variable`	Tightened it to match only real equations
Broad questions always refused	A pointed "do these passages answer this?" check can't pass for "explain everything"	**Overview mode** that reads the whole named document
Slow answers	Two model calls per question	Skip the check on confident hits: **half the model calls**
Evaluation reported MRR = 0.00	Retrieval results lacked the exact document ID the scorer matched on	Returned `doc` with every hit, plus a regression test
"teach me it" refused	Only Teach Me sent the conversation, and the search used the literal words	Conversation sent in every mode, plus a **query rewriter**
Bot repeated itself after "yes it does"	The model saw only 700 characters of its last reply, and "yes" was misread	Detect understood / not understood; pass the whole last reply; anti-repeat checklist
"osimodel" refused	A joined word wrecked the semantic match	**Spelling repair** against the notes' vocabulary
Ollama silently truncating notes	Default context window too small	Set the context window to **8k tokens**
Private files exposed in a public repo	`users.db` and uploads were tracked by Git	`private/` folder: owner-only, Git-ignored, auto-migration
Startup took many manual steps	Docker, Qdrant, Ollama, venv, uvicorn and the tunnel started by hand	`vstart`: one command brings everything up in order

17. Vision vs. Delivered

Original vision (June)	Final product (October)
Answer from the student's own material	✓ With citations to slide/page, per student
Honest when unsure	✓ Three-band gate, honest refusals, closest-topic suggestions, no invented reasons
Escalate to a teacher/senior; answers feed back into the knowledge base	✓ Ask a Helper, with transparent routing and the feedback loop
Adaptive teaching	✓ Teach Me: one idea at a time, adapts to "yes / no / next"
Exam practice with rubric feedback	✓ Question generation and four-part feedback
Any format, including handwriting	✓ 8+ formats with OCR fallback and caching
Runs locally, private	✓ Local LLM, private folder, per-user isolation
(not planned)	**+** Natural conversation layer, typo repair, evaluation harness, one-command startup, public demo link
Fine-tuning on course data	⏳ Future work
Grading handwritten answers	⏳ Future work

18. Limitations

- **8B model limits:** the small local model occasionally over-explains or draws a conclusion the notes don't state. Prompts and checklists reduce this but can't fully remove it.
- **Latency:** about 2–4 s for quick answers, longer for whole-module walkthroughs, on a laptop with no GPU server.
- **OCR** for handwriting needs a vision model, which is cloud-based when used.
- **Single-machine deployment:** the demo link changes on every restart (Cloudflare quick tunnel).

---

19. Future Work

1. **Fine-tune** a small model (LoRA) on course material and helper answers.
2. **Handwritten answer grading** (photo of an answer sheet → OCR → grounded feedback).
3. **Graph-based retrieval** across modules for "compare X in module 1 with Y in module 3".
4. **Spaced-repetition practice** built from the student's weak topics.
5. **Malayalam and Hindi** question support (the tokeniser is already Indic-aware).
6. A permanent hosted deployment with a fixed domain.

---

20. Live Demo Script (≈ 5 minutes)

1. **Start:** `vstart`: Docker → Qdrant → Ollama → app → public link, in one command.
2. **Greeting:** "hi" gives a friendly reply naming topics from the uploaded notes.
3. **Grounded answer:** "What is the OSI model?" gives a cited answer; click a citation chip.
4. **Overview:** "explain each concept of module 2 in detail" gives a structured walkthrough of the whole module.
5. **Follow-up:** "teach me the transport layer" → "ok" → it moves on to the next idea → "I don't get it" → it re-explains more simply.
6. **Typo:** "explain the osimodel" → "understood as: explain the osi model".
7. **Honesty:** "Explain photosynthesis" → "I don't have that in the material", plus the closest topics the notes cover.
8. **Ask a Helper:** on a tentative answer, open Ask a Helper, see who it goes to, send it; answer as the helper; ask again, and it now answers from "Helper answers".
9. **Exam & Practice:** generate 5 questions, then grade a typed answer.
10. **Privacy:** sign in as another student, who can't see the first student's notes.

---

21. Conclusion

Vidhyāsathi shows that a **small, local model can be a trustworthy study companion** when it is wrapped in the right engineering: - **retrieval** that stays inside the student's own material, - a **confidence gate** that prefers honesty to fluency, - a **conversation layer** that understands how students actually talk, - a **human in the loop** for what the notes can't answer, with a loop that makes the system smarter over time.

Answer when grounded. Be honest when unsure. Escalate when it matters.

---

22. References

1. A. Singh et al., "Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG," arXiv:2501.09136, 2025.
2. S.-Q. Yan et al., "Corrective Retrieval Augmented Generation (CRAG)," arXiv:2401.15884, 2024.
3. S. Jeong et al., "Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity," NAACL 2024, arXiv:2403.14403.
4. D. Edge et al., "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," arXiv:2404.16130, 2024.
5. D. Ru et al., "RAGChecker: A Fine-grained Framework for Diagnosing Retrieval-Augmented Generation," NeurIPS 2024, arXiv:2408.08067.
6. L. Huang et al., "A Survey on Hallucination in Large Language Models," ACM TOIS 2024, arXiv:2311.05232.
7. S. Wang et al., "Large Language Models for Education: A Survey and Outlook," arXiv:2403.18105, 2024.
8. Z. Chu et al., "Enhancing LLM-Based Short Answer Grading with Retrieval-Augmented Generation," arXiv:2504.05276, 2025.

Appendix — Key Numbers at a Glance

- **8** research papers · **8+** file formats · **3** study modes · **1** human-in-the-loop path
- **3-band** confidence gate: 0.30 / 0.45 / 0.55
- Chunks of **800** characters (150 overlap) · top **15** retrieved → best **5** after re-ranking
- **384-dim** MiniLM embeddings · **8k-token** context window · **Llama 3.1 8B** local
- **453** backend tests · **186** UI tests · **3** QA rounds · **18/18** live checks
- **100%** honest refusals · **75%** judged accuracy · **≈ 2–4 s** per answer
- **1 command** to start (`vstart`), **1** to stop (`vstop`), **1** to check everything (`vcheck`)